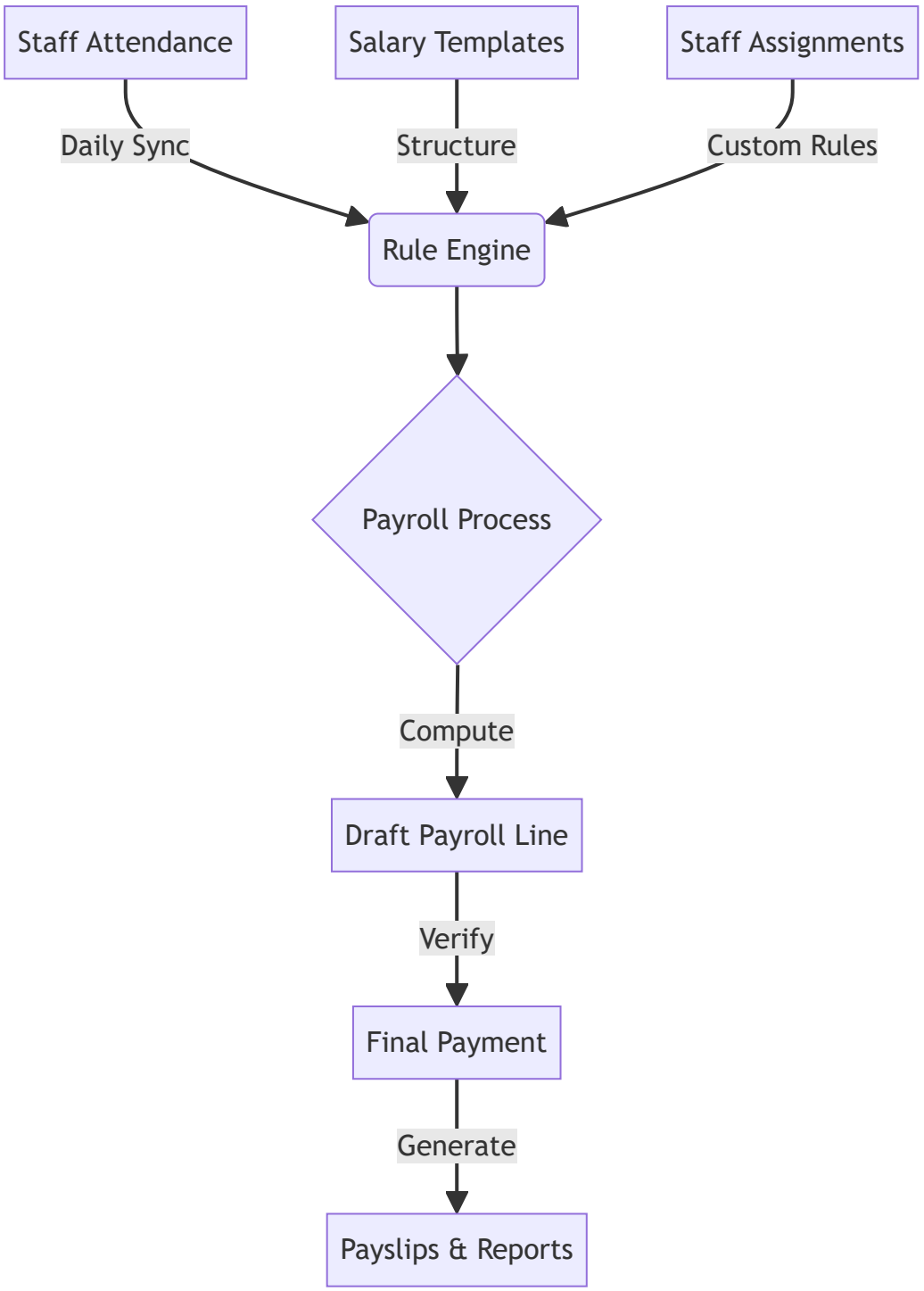
Thinktech Attendance - Payroll System Documentation

This document provide a detailed technical overview of the Payroll System implemented in the Thinktech Attendance platform.

1. System Overview

The Payroll system is an automated engine that synchronizes attendance data, organizational policies, and statutory compliance rules to derive accurate employee compensation.



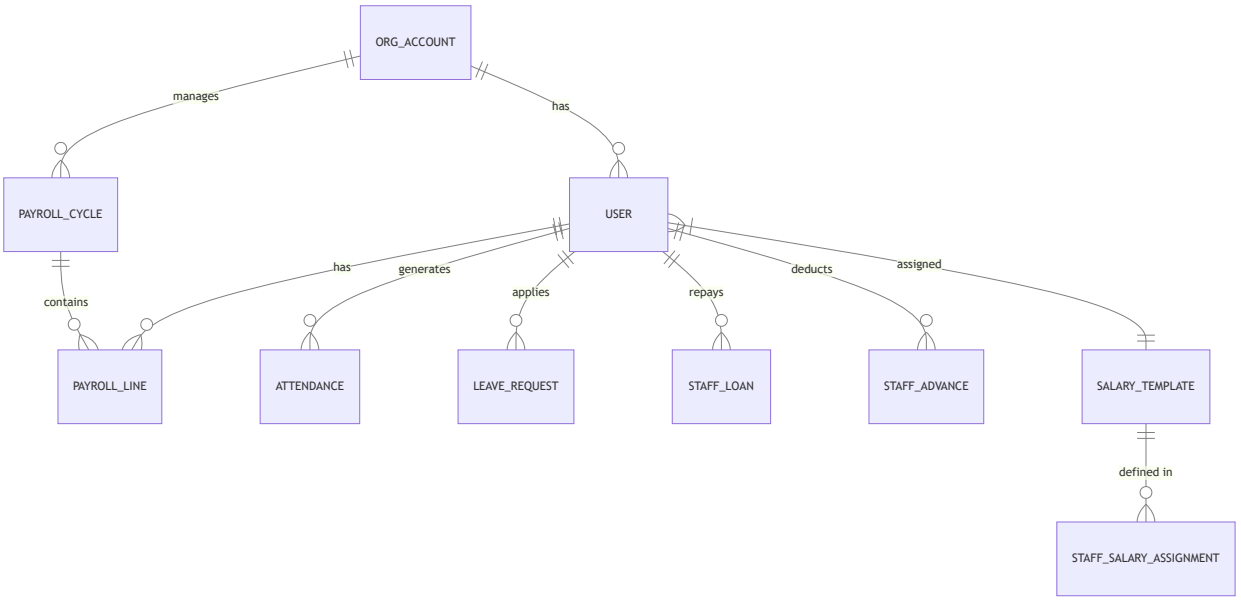
The payroll system is designed to automate salary calculations based on attendance records, shift schedules, and organizational policies. It supports two main modes:

- **Live Compute (Projected)**: Real-time salary estimation for the current month.
- **Generated Records (Historical)**: Locked-in payroll cycles for past months, stored as snapshots to ensure data integrity and persistence for payslip generation.

---

2. Entity Relationship Diagram (ERD)

The following diagram illustrates the core models involved in the payroll process.



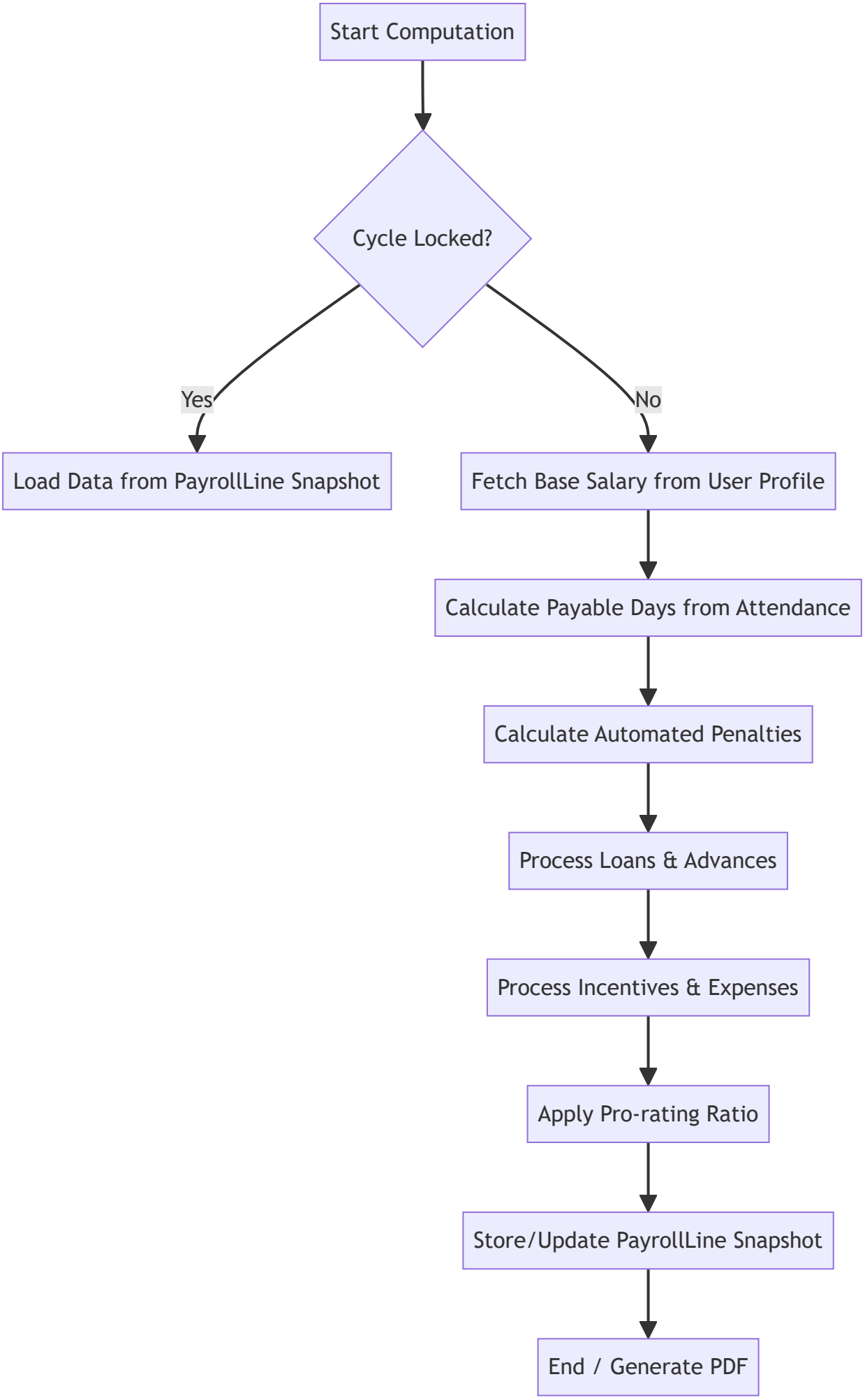
Key Models Descriptions:

- **PayrollCycle**: Represents a specific month's payroll (e.g., `2024-03`). Tracks status (`DRAFT`, `PAID`) and payment dates.
- **PayrollLine**: A snapshot of a specific user's salary metadata for a cycle. Stores earnings, deductions, incentives, and attendance summaries as JSON blobs.
- **SalaryTemplate**: Defines the structure of earnings and deductions (e.g., Basic = 50% of Gross, PF = 12% of Basic).
- **Attendance**: The source of truth for working days, overtime, and penalty minutes.
- **StaffLoan / StaffAdvance**: Specific financial deductions applied during the calculation phase.

---

3. Salary Calculation Flow

The calculation logic is centralized in `backend/src/services/payrollService.js`.


Start Computation
Cycle Locked?
Yes
No
Load Data from PayrollLine Snapshot
Fetch Base Salary from User Profile
Calculate Payable Days from Attendance
Calculate Automated Penalties
Process Loans & Advances
Process Incentives & Expenses
Apply Pro-rating Ratio
Store/Update PayrollLine Snapshot
End / Generate PDF

Logic Steps:

1. **Payable Days Calculation**:
   - `Payable Units = Present Days + (Half Days * 0.5) + Paid Leave + Weekly Offs + Holidays`.
   - `Ratio = Payable Units / Total Days in Month`.
2. **Automated Penalties**:
   - **Late Punch-In**: Based on `LatePunchInRule`, deducts a fixed amount or daily salary percentage if frequency exceeds threshold.
   - **Early Exit**: Based on `EarlyExitRule`, applies penalties for leaving before shift completion.
   - **Break Penalty**: Penalizes excess break duration beyond allowed limits.
3. **Pro-rating Earnings**:
   - Fixed earnings (HRA, Conveyance) are multiplied by the `Ratio`.
   - Variable earnings (Overtime, Early OT) are added as absolute values.

---

4. API Reference

Cycle Management

- `GET /admin/payroll?monthKey=YYYY-MM`: Fetch all payroll lines for a specific month.
- `POST /admin/payroll/:cycleId/compute`: Trigger a full re-computation of all staff in the cycle.
- `POST /admin/payroll/manual-historical`: Manually add a historical record (bypass auto-calc).

Payments

- `POST /admin/payroll/:cycleId/lines/mark-paid`: Bulk update selected lines to `PAID` status.
- `POST /admin/payroll/:cycleId/mark-paid`: Update the entire cycle status to `PAID`.

Exports

- `GET /admin/payroll/:cycleId/export`: Download cycle data as CSV.
- `GET /admin/payroll/:cycleId/salary-register-excel`: Detailed Excel report with group-wise (Dept/Designation) totals.
- `GET /admin/payroll/monthly-summary-excel`: High-level summary report.

---

5. Automated Penalty Logic Details

Late Punch-In Penalty

Feature	Implementation
Detection	Compares `punchedInAt` with `assignedShift.startTime` + buffer.
Tiers	Configured in `AttendanceAutomationRule`. Multiple frequency tiers supported (e.g., 3rd late in month = 0.5 day deduction).
Execution	Calculated during `computeLatePenaltyMeta` in `payrollService.js`.

Early Exit Penalty

- Triggers if `punchedOutAt` is significantly earlier than `assignedShift.endTime`.
- Grace periods and specific penalty amounts/days can be set per org.

---

6. Financial Adjustments

Loans (EMI)

- Deduction is applied automatically if the current `monthKey` is within the loan's tenure range.
- `Total Loan EMI = Sum(Active Loans EMI where Month is within Start + Tenure)`.

Advances

- Single-month deductions or split-month deductions.
- Deducted from the Net Salary after all other earnings/deductions are calculated.

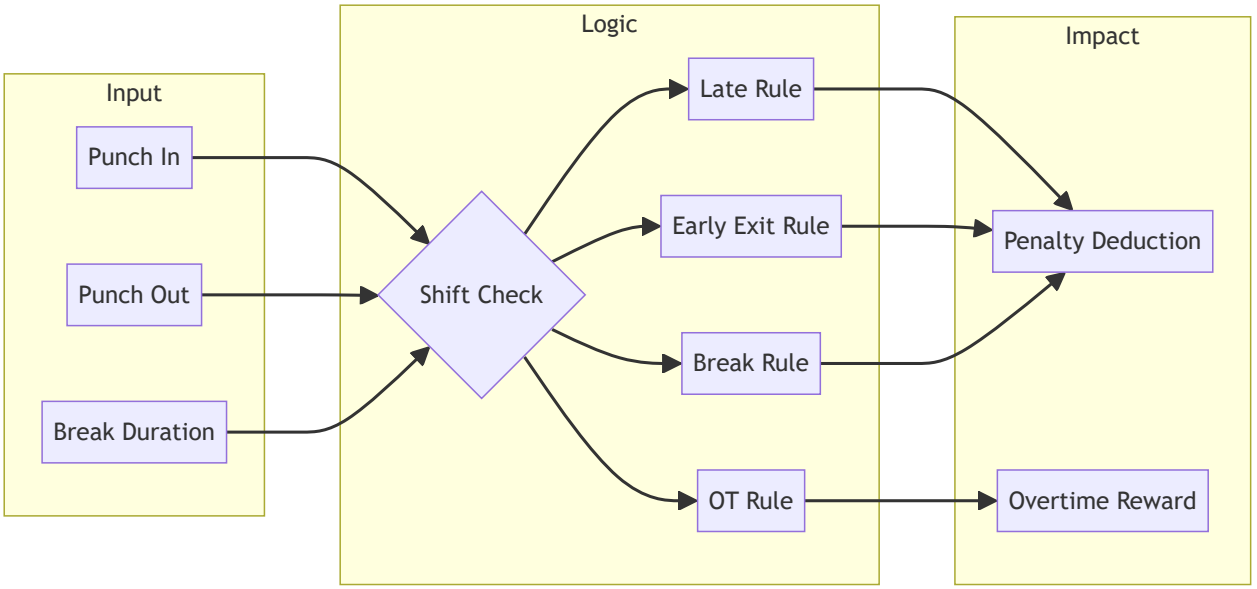
Expense Reimbursements

- Fetches all `ExpenseClaim` records with `status: 'settled'` for the month.
- Added as absolute earnings (Tax-free by default).

---

7. Automation Rules Engine

The system uses a flexible rules engine to handle attendance-based earnings and deductions. Rules are assigned via **Assignments** (User-level) or **Org-level Defaults**.



7.1 Late Punch-In Penalty (LatePunchInRule)

- **Logic**: Compares `punchedInAt` against the assigned `shift.startTime` plus a buffer.
- **Deduction Types**:
  - `SLABS`: Tiered deductions based on minutes late (e.g., 1-15 mins = X deduction, 16-30 mins = Y). Supports frequency (e.g., 3rd time in a month applies a deduction).
  - `FIXED_AMOUNT`: A flat penalty if the threshold is crossed.
  - `HALF_DAY` / `FULL_DAY`: Deducts partial or entire daily salary.
  - `SALARY_MULTIPLIER`: Deducts based on an hourly rate multiplier (Hourly Rate = Daily Salary / 8).
- **Service**: Managed by `latePunchInService.js`.

7.2 Early Exit Penalty (EarlyExitRule)

- **Logic**: Triggers if `punchedOutAt` is earlier than the assigned `shift.endTime`.
- **Key Features**:
  - **Thresholds**: Defined in minutes (e.g., leaving 30 mins early triggers a penalty).
  - **Deductions**: Can be configured as `FIXED_AMOUNT` or `SALARY_MULTIPLIER`.
  - **Half/Full Day Overrides**: Higher priority flags that deduct 0.5 or 1 day salary if the early exit duration exceeds specific thresholds.
- **Service**: Managed by `earlyExitService.js`.

7.3 Break Management (BreakRule)

- **Logic**: Monitors the total duration spent on "Break" status during a shift.

- **Calculation**:
  - `Excess Minutes = Total Break Minutes - Allowed Threshold`.
  - Penalty is applied based on the highest applicable threshold tier.
- **Supported Deductions**: `FIXED_AMOUNT`, `SALARY_MULTIPLIER` (Hourly rate x Multiplier), or Half/Full Day deductions.
- **Service**: Managed by `breakService.js`.

7.4 Overtime Logic (OvertimeRule)

- **Calculation Types**:
  - `POST_PAYABLE_HOURS`: Overtime starts after total work hours exceed a threshold (e.g., 9 hours).
  - `SHIFT_END`: Overtime starts from the moment the assigned shift ends.
  - `POST_AND_END`: Requires BOTH total hours met AND shift boundary crossed.
- **Reward Mechanics**:
  - Supports tiered value (e.g., first 2 hours at 1.5x, then 2.0x).
  - Reward can be `FIXED_AMOUNT_PER_HOUR` or `SALARY_MULTIPLIER` (Hourly Rate x Multiplier).
- **Service**: Managed by `overtimeService.js`.

7.5 Early Overtime (EarlyOvertimeRule)

- **Logic**: Rewards work performed before the assigned `shift.startTime`.
- **Calculation**: `Shift Start - Punch In Time`.
- **Eligibility**: Only counts if the early work exceeds a minimum threshold (e.g., must come at least 30 mins early to count).
- **Reward**: Same tiered structure as standard Overtime.
- **Service**: Managed by `earlyOvertimeService.js`.

7.6 Tenure Bonus System (TenureBonusRule)

- **Logic**: Automated loyalty rewards based on the employee's `date_of_joining`.
- **Brackets**: Configured as JSON ranges (e.g., 12-24 months = 5% of Basic, 24-36 months = 10%).
- **Execution**: The `payrollService.js` calculates tenure months as of the target month end and applies the corresponding percentage bracket to the current month's base earnings.
- **Earning Type**: Added to the payslip as `TENURE_BONUS`.

> [!NOTE] All automated calculations use **(Basic Salary + DA)** as the standard `Base Salary` for hourly rate derivations unless specified otherwise in the Salary Template.